

docs-ari-rfc-process

ARI RFC Process

When you need an RFC

Lifecycle

RFC template

Principles

ARI RFC Process

How changes to the [ARI specification](#) are proposed, discussed, and accepted. The point of a written process is that **the rules for changing the standard are themselves on the record** — adopters need to trust that the contract won't shift under them arbitrarily.

This process governs the *spec*. Changes to the *implementation* (marrow) follow normal PRs and [STABILITY.md](#).

When you need an RFC

Change	RFC required?
Type / clarification that doesn't change meaning	No — normal PR
New OPTIONAL field, new profile, new informative appendix	Lightweight RFC
Changing a MUST/SHOULD, tightening an existing requirement, removing anything	Full RFC
Anything that could make a currently-conformant runtime non-conformant	Full RFC + migration note

Lifecycle

Draft → Proposed → Accepted → Stable (or → Rejected / Withdrawn)

issue/PR	RFC doc	maintainer	ships in a
sketch	+ discuss	sign-off	versioned spec

1. **Draft** — open a GitHub issue tagged ari-spec describing the problem. Problem first, solution second. "What breaks today / what can't be expressed" beats "here's my API."
2. **Proposed** — submit an RFC document (PR adding docs/rfcs/NNNN-title.md) using the template below. Discussion happens on the PR.
3. **Accepted** — a maintainer (today) or the working group (post-[governance](#) trigger 2) signs off, with a recorded rationale. Acceptance requires: at least one reference-implementation sketch or a clear conformance-kit test for the new behavior.
4. **Stable** — the change ships in a versioned spec release and the [conformance kit](#) is updated to cover it. From here, [ARI §11](#) backward-compatibility rules apply.

Rejected/withdrawn RFCs are kept (not deleted) so the reasoning is preserved.

RFC template

```
# RFC NNNN: <title>

- Status: Proposed
- Author(s): <name>
- Affects: ARI §<section(s)>
- Profiles: Core | Embedded | Server

## Summary
One paragraph.

## Motivation
What's broken or inexpressible today? Who is hurt and how?

## Specification
The exact normative change (MUST/SHOULD/MAY), as it would read in
ARI-SPEC.md.

## Conformance impact
- New/changed conformance-kit checks
- Does this make any currently-conformant runtime non-conformant? If
so, migration.

## Reference implementation
Link to an marrow sketch/PR, or describe how it'd be implemented.

## Alternatives considered
What else, and why not.

## Backward compatibility
Additive? Breaking? Deprecation window?
```

Principles

- **Additive over breaking.** Within a major version, prefer new optional fields and new profiles to changing existing requirements.
- **Spec follows running code.** Prefer changes proven in a real implementation (ideally a real deployment) over speculative design.
- **Conformance is the contract.** If a change can't be expressed as a conformance-kit check, it probably isn't a crisp enough requirement.